WORK INTEGRATED LEARNING PROGRAMME

M.Tech. in Artificial Intelligence & Machine Learning

FIRST SEMESTER

ASSIGNMENT 2

AIMLZG565: Machine Learning

Student Name	Manish Raj
BITS ID	**2026AJ05499**

1. GitHub Repository

https://github.com/officialinmailbox-art/Assignment_Bits/tree/ML_Assignment

2. Live Streamlit Application

https://assignmentbits-ngaeuo3thxpnluqysgj2iz.streamlit.app/

3. Screenshots – Execution on BITS Virtual Lab

This section is intended to contain screenshots demonstrating execution of the Machine Learning Assignment on the BITS Virtual Lab. Attach the relevant execution screenshots with the submission.

4. README.md – Project Documentation

• **Project:** Machine Learning Classification on the Adult (Census Income) Dataset.

• **Objective:** Predict whether annual income is greater than $50K (1) or $50K or less (0).

• **Algorithms:** Logistic Regression, Decision Tree, K-Nearest Neighbours (KNN), Gaussian Naive Bayes and Support Vector Machine (SVM with RBF kernel).

• **Evaluation metrics:** Accuracy, AUC, Precision, Recall, F1 Score and Matthews Correlation Coefficient (MCC).

• **Data preparation:** Missing-value cleaning, numeric feature scaling, categorical one-hot encoding and an 80:20 stratified train-test split.

• **Results:** Decision Tree achieved Accuracy 0.8524, F1 Score 0.6700 and MCC 0.5820. It was the overall selected model, with an average AUC, F1 and MCC score of 0.7175.

• **Application:** Streamlit application for evaluating trained classification models using an uploaded test dataset.

How to run: Install dependencies from requirements.txt, train/save the models as applicable, then run `streamlit run streamlit_app.py`.

Submitted as part of the First Semester coursework for AIMLZG565 – Machine Learning.

```python
# ============================================================================
# Machine Learning Classification on the Adult (Census Income) Dataset
# Models : Logistic Regression, Decision Tree, KNN, Naive Bayes, SVM
# Metrics: Accuracy, AUC, Precision, Recall, F1, MCC
# Goal   : Predict whether a person earns >50K per year (1) or <=50K (0)
# ============================================================================

# ----------------------------------------------------------------------------
# STEP 1: Import required libraries
# ----------------------------------------------------------------------------
import pandas as pd                                          # data loading and
        tables
from sklearn.model_selection import train_test_split  # split data into
        train/test
from sklearn.compose import ColumnTransformer         # apply different
        preprocessing per column type
from sklearn.pipeline import Pipeline                 # chain preprocessing
        + model together
from sklearn.preprocessing import StandardScaler, OneHotEncoder


# The 5 classification models
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC


# Evaluation metrics
from sklearn.metrics import (accuracy_score, roc_auc_score, precision_score,
                             recall_score, f1_score, matthews_corrcoef)


# ----------------------------------------------------------------------------
# STEP 2: Load the dataset
# ----------------------------------------------------------------------------
df = pd.read_csv("adult_full.csv")   # the CSV created from the zip file
print("Original shape:", df.shape)


# ----------------------------------------------------------------------------
```

```
# STEP 3: Data cleaning
# ---------------------------------------------------------------------
# Missing values in this dataset are written as '?', so convert them to
# proper missing values and remove those rows.
df = df.replace("?", pd.NA).dropna()
print("Shape after removing missing values:", df.shape)

# Convert the target column from text to numbers:
#   '>50K'  -> 1 (positive class)
#   '<=50K' -> 0 (negative class)
df["income"] = (df["income"] == ">50K").astype(int)

# ---------------------------------------------------------------------
# STEP 4: Separate features (X) and target (y)
# ---------------------------------------------------------------------
X = df.drop(columns="income")   # all input columns
y = df["income"]                # the column we want to predict

# Identify numeric and categorical columns (they need different
        preprocessing)
num_cols = X.select_dtypes(include="number").columns.tolist()
cat_cols = X.select_dtypes(exclude="number").columns.tolist()
print("Numeric columns    :", num_cols)
print("Categorical columns:", cat_cols)

# ---------------------------------------------------------------------
# STEP 5: Split into training and testing sets (80% / 20%)
# ---------------------------------------------------------------------
# stratify=y keeps the same >50K / <=50K ratio in both sets.
# random_state=42 makes the split reproducible.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("Train size:", X_train.shape, "| Test size:", X_test.shape)

# ---------------------------------------------------------------------
# STEP 6: Build the preprocessing step
```

```
# ---------------------------------------------------------------------
# - Numeric columns    : standardize (mean 0, std 1) so large-valued columns
#                        (e.g. fnlwgt) do not dominate KNN / SVM / Logistic
        Regression
# - Categorical columns: one-hot encode into 0/1 columns
#                        (handle_unknown='ignore' avoids errors on unseen
        categories)
def make_preprocessor():
    return ColumnTransformer([
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
        cat_cols),
    ])


# ---------------------------------------------------------------------
# STEP 7: Define the 5 models
# ---------------------------------------------------------------------
models = {
    # 1. Linear model that predicts class probability using the sigmoid
        function
    "Logistic Regression": LogisticRegression(max_iter=1000),

    # 2. Tree of if/else rules; max_depth limits growth to prevent
        overfitting
    "Decision Tree": DecisionTreeClassifier(max_depth=8, random_state=42),

    # 3. Classifies a point by majority vote of its 7 nearest neighbours
    "KNN": KNeighborsClassifier(n_neighbors=7),

    # 4. Probabilistic model based on Bayes' theorem (assumes Gaussian
        features)
    "Naive Bayes (Gaussian)": GaussianNB(),

    # 5. Finds the best separating boundary; RBF kernel handles non-linear
        data
    "SVM (RBF)": SVC(kernel="rbf", C=1.0),
}


# ---------------------------------------------------------------------
# STEP 8: Train each model and calculate the evaluation metrics
# ---------------------------------------------------------------------
```

```python
results = []   # will hold one row of metrics per model


for name, clf in models.items():


    # Pipeline = preprocessing + model, so test data gets exactly the same
    # transformation that was learned from the training data (no data
        leakage).

    pipe = Pipeline([("pre", make_preprocessor()), ("clf", clf)])


    # 8a. Train the model

    pipe.fit(X_train, y_train)


    # 8b. Predict class labels (0 / 1) for the test set

    y_pred = pipe.predict(X_test)


    # 8c. Get continuous scores, which AUC needs.
    #     Most models give probabilities; SVC gives a decision-function
        score.

    if hasattr(pipe, "predict_proba"):
        y_score = pipe.predict_proba(X_test)[:, 1]   # probability of class
        1

    else:

        y_score = pipe.decision_function(X_test)


    # 8d. Calculate all six metrics

    results.append({

        "Model":     name,
        "Accuracy":  accuracy_score(y_test, y_pred),    # % of correct
        predictions
        "AUC":       roc_auc_score(y_test, y_score),    # ability to rank
        positives above negatives
        "Precision": precision_score(y_test, y_pred),   # of predicted >50K,
        how many are truly >50K
        "Recall":    recall_score(y_test, y_pred),      # of actual >50K,
        how many were found
        "F1":        f1_score(y_test, y_pred),          # harmonic mean of
        precision and recall
        "MCC":       matthews_corrcoef(y_test, y_pred), # balanced score,
        good for imbalanced data

    })

    print(f"Finished: {name}")
```

```python
# ---------------------------------------------------------------------
# STEP 9: Show and save the comparison table
# ---------------------------------------------------------------------
results_df = pd.DataFrame(results).set_index("Model").round(4)
print("\n=========== Model Comparison ===========")
print(results_df.to_string())

results_df.to_csv("model_results.csv")
print("\nResults saved to model_results.csv")

# ---------------------------------------------------------------------
# STEP 10: Identify the best model for each metric
# ---------------------------------------------------------------------
print("\nBest model per metric:")
for metric in results_df.columns:
    print(f"  {metric:<10}: {results_df[metric].idxmax()}
        ({results_df[metric].max():.4f})")
```

```
Original shape: (48842, 15)
Shape after removing missing values: (45222, 15)
Numeric columns    : ['age', 'fnlwgt', 'education_num', 'capital_gain',
'capital_loss', 'hours_per_week']
Categorical columns: ['workclass', 'education', 'marital_status',
'occupation', 'relationship', 'race', 'sex', 'native_country']
Train size: (36177, 14) | Test size: (9045, 14)
Finished: Logistic Regression
Finished: Decision Tree
Finished: KNN
Finished: Naive Bayes (Gaussian)
Finished: SVM (RBF)

========== Model Comparison ==========
                       Accuracy     AUC  Precision  Recall      F1     MCC
Model
Logistic Regression       0.8450  0.9020     0.7344  0.5870  0.6525  0.5601
Decision Tree             0.8524  0.9004     0.7515  0.6044  0.6700  0.5820
KNN                       0.8302  0.8735     0.6781  0.5995  0.6364  0.5278
Naive Bayes (Gaussian)    0.6193  0.8389     0.3877  0.9242  0.5462  0.3891
SVM (RBF)                 0.8482  0.8957     0.7513  0.5794  0.6542  0.5668

Results saved to model_results.csv

Best model per metric:
  Accuracy  : Decision Tree (0.8524)
  AUC       : Logistic Regression (0.9020)
  Precision : Decision Tree (0.7515)
  Recall    : Naive Bayes (Gaussian) (0.9242)
  F1        : Decision Tree (0.6700)
  MCC       : Decision Tree (0.5820)
```

```
# ---------------------------------------------------------------------
# STEP 11: OBSERVATIONS ON MODEL PERFORMANCE
# ---------------------------------------------------------------------

observations = {
    "Logistic Regression": (
        "Logistic Regression provides a good baseline for income "
        "classification. It is computationally efficient and identifies "
        "relationships between demographic features and income. However, "
        "it may not capture complex non-linear relationships."
    ),

    "Decision Tree": (
        "Decision Tree identifies complex patterns and non-linear "
        "relationships between features. It is easy to interpret and "
        "understand. The maximum depth of 8 helps reduce overfitting."
    ),

    "KNN": (
        "KNN classifies individuals using their seven nearest neighbours. "
        "StandardScaler improves distance-based calculations. However, "
        "high-dimensional categorical features may affect its performance, "
        "and prediction can be computationally expensive."
    ),

    "Naive Bayes (Gaussian)": (
        "Gaussian Naive Bayes is fast and uses probability-based "
        "classification. However, its assumptions of conditional "
        "independence and Gaussian-distributed features may not accurately "
        "represent all features in the Adult dataset."
    ),

    "SVM (RBF)": (
        "SVM with an RBF kernel can identify complex non-linear decision "
        "boundaries. It can provide good classification performance "
```

```
        "but requires more computational time and memory for larger
        datasets."
    )
}

print("\n" + "=" * 80)
print("       OBSERVATIONS ON MODEL PERFORMANCE")
print("=" * 80)

for model_name, observation in observations.items():

    print("\nML Model Name:", model_name)
    print("Observation:", observation)

    if model_name in results_df.index:

        print("\nActual Evaluation Metrics:")

        for metric in results_df.columns:
            print(
                f"{metric:<12}: "
                f"{results_df.loc[model_name, metric]:.4f}"
            )

        print("-" * 80)


# ------------------------------------------------------------------
# STEP 12: IDENTIFY OVERALL WINNER
# ------------------------------------------------------------------

print("\n" + "=" * 80)
print("                 OVERALL MODEL WINNER")
print("=" * 80)

# Calculate average of AUC, F1 and MCC
overall_scores = results_df[["AUC", "F1", "MCC"]].mean(axis=1)
```

```
# Identify winner
winner = overall_scores.idxmax()
winner_score = overall_scores.max()

print("\nModel Performance Based on AUC, F1 and MCC:")

print(overall_scores.round(4).to_string())

print("\nOverall Winner:", winner)
print("Average Score:", round(winner_score, 4))



# ------------------------------------------------------------------------
# STEP 13: GENERATE ASSIGNMENT CONCLUSION
# ------------------------------------------------------------------------

print("\n" + "=" * 80)
print("                  ASSIGNMENT CONCLUSION")
print("=" * 80)

print("""
The Adult Census Income Dataset was used to implement five machine
learning classification algorithms.

All models were trained and evaluated using the same 80:20
training-testing split and preprocessing pipeline.

The models were compared using the following evaluation metrics:

1. Accuracy
2. AUC Score
3. Precision
4. Recall
5. F1 Score
6. Matthews Correlation Coefficient (MCC)
```

```
""")

print(
    f"The overall best-performing model is {winner}, "
    f"with an average AUC, F1 and MCC score of {winner_score:.4f}."
)

print("""
The winning model was selected based on the average of AUC, F1
and MCC scores. This provides a more balanced evaluation than
accuracy alone, particularly for an imbalanced dataset.

Therefore, the selected model is considered the most suitable
among the five implemented classifiers for the Adult Census
Income Classification task.
""")

# ------------------------------------------------------------------
# STEP 14: SAVE OBSERVATIONS AND CONCLUSION TO A TEXT FILE
# ------------------------------------------------------------------

# Create results folder if it does not exist
os.makedirs("results", exist_ok=True)

# Save observations
with open("results/model_observations.txt", "w", encoding="utf-8") as file:

    file.write("OBSERVATIONS ON MODEL PERFORMANCE\n")
    file.write("=" * 80 + "\n\n")

    for model_name, observation in observations.items():

        file.write("ML Model Name: " + model_name + "\n")
        file.write("Observation: " + observation + "\n\n")

        if model_name in results_df.index:
```

```
            file.write("Evaluation Metrics:\n")

            for metric in results_df.columns:
                file.write(
                    f"{metric}: {results_df.loc[model_name, metric]:.4f}\n"
                )

        file.write("\n" + "-" * 80 + "\n")

    file.write("\nOVERALL WINNER\n")
    file.write("=" * 80 + "\n")
    file.write(f"Winner: {winner}\n")
    file.write(f"Average Score: {winner_score:.4f}\n")

print("Observations saved successfully!")
print("File location: results/model_observations.txt")
```

```
================================================================================
       OBSERVATIONS ON MODEL PERFORMANCE
================================================================================

ML Model Name: Logistic Regression
Observation: Logistic Regression provides a good baseline for income
classification. It is computationally efficient and identifies relationships
between demographic features and income. However, it may not capture complex
non-linear relationships.

Actual Evaluation Metrics:
Accuracy    : 0.8450
AUC         : 0.9020
Precision   : 0.7344
Recall      : 0.5870
F1          : 0.6525
MCC         : 0.5601
--------------------------------------------------------------------------------

ML Model Name: Decision Tree
Observation: Decision Tree identifies complex patterns and non-linear
relationships between features. It is easy to interpret and understand. The
maximum depth of 8 helps reduce overfitting.

Actual Evaluation Metrics:
Accuracy    : 0.8524
AUC         : 0.9004
Precision   : 0.7515
Recall      : 0.6044
F1          : 0.6700
MCC         : 0.5820
--------------------------------------------------------------------------------

ML Model Name: KNN
```

Observation: KNN classifies individuals using their seven nearest neighbours. StandardScaler improves distance-based calculations. However, high-dimensional categorical features may affect its performance, and prediction can be computationally expensive.

Actual Evaluation Metrics:

Accuracy : 0.8302

AUC : 0.8735

Precision : 0.6781

Recall : 0.5995

F1 : 0.6364

MCC : 0.5278

ML Model Name: Naive Bayes (Gaussian)

Observation: Gaussian Naive Bayes is fast and uses probability-based classification. However, its assumptions of conditional independence and Gaussian-distributed features may not accurately represent all features in the Adult dataset.

Actual Evaluation Metrics:

Accuracy : 0.6193

AUC : 0.8389

Precision : 0.3877

Recall : 0.9242

F1 : 0.5462

MCC : 0.3891

ML Model Name: SVM (RBF)

Observation: SVM with an RBF kernel can identify complex non-linear decision boundaries. It can provide good classification performance but requires more computational time and memory for larger datasets.

```
Actual Evaluation Metrics:
Accuracy    : 0.8482
AUC         : 0.8957
Precision   : 0.7513
Recall      : 0.5794
F1          : 0.6542
MCC         : 0.5668
--------------------------------------------------------------------------------

================================================================================
                 OVERALL MODEL WINNER
================================================================================

Model Performance Based on AUC, F1 and MCC:
Model
Logistic Regression       0.7049
Decision Tree             0.7175
KNN                       0.6792
Naive Bayes (Gaussian)    0.5914
SVM (RBF)                 0.7056

Overall Winner: Decision Tree
Average Score: 0.7175

================================================================================
                  ASSIGNMENT CONCLUSION
================================================================================

The Adult Census Income Dataset was used to implement five machine
learning classification algorithms.

All models were trained and evaluated using the same 80:20
training-testing split and preprocessing pipeline.

The models were compared using the following evaluation metrics:
```

```
1. Accuracy
2. AUC Score
3. Precision
4. Recall
5. F1 Score
6. Matthews Correlation Coefficient (MCC)

The overall best-performing model is Decision Tree, with an average AUC, F1
and MCC score of 0.7175.

The winning model was selected based on the average of AUC, F1
and MCC scores. This provides a more balanced evaluation than
accuracy alone, particularly for an imbalanced dataset.

Therefore, the selected model is considered the most suitable
among the five implemented classifiers for the Adult Census
Income Classification task.

Observations saved successfully!
File location: results/model_observations.txt
```